

Python en Terminale

Le Python attendu en spécialités **mathématiques** et **physique-chimie** : des bases du langage jusqu'aux usages courants de `numpy` et `matplotlib`.



Accéder à la webapp

Scanne le QR code ou ouvre le lien dans un navigateur récent (ordinateur, tablette ou téléphone). **Rien à installer, aucun compte élève à créer** : Python s'exécute directement dans la page, et tes réponses restent enregistrées dans ce navigateur.

<https://ressources.cours-sciences.fr/python-lycee/>

Premier chargement : environ 25 Mo, quelques secondes en Wi-Fi. Le voyant « Python prêt » passe au vert quand tout est disponible.

Déroulé

DÉBUT	DURÉE	ÉTAPE	EXOS	FAIT
0'	5 min	0. Prise en main Écrire et exécuter sa première ligne de Python.	1	☐
5'	10 min	1. Variables et calculs Stocker des valeurs, calculer, connaître les types de base.	2	☐
15'	15 min	2. Fonctions et conditions Écrire une fonction qui renvoie un résultat, utiliser if / elif / else.	2	☐
30'	15 min	3. Boucles et suites Répéter avec for, chercher un seuil avec while, calculer les termes d'une suite.	3	☐
45'	10 min	4. Listes Créer, parcourir et transformer une liste ; listes en compréhension.	3	☐
55'	15 min	5. NumPy Créer des tableaux, calculer sans boucle, exploiter des mesures.	3	☐
70'	12 min	6. Tracer avec Matplotlib Tracer une courbe ou des points, légender un graphique, représenter des vecteurs.	2	☐
82'	8 min	7. Synthèse : méthode d'Euler Combiner boucle, tableaux et tracé pour résoudre une équation différentielle.	1	☐
90'		Fin du parcours · bonus : dichotomie, bac à sable	17	

1 • Lire

la notion, courte, avec un exemple.

2 • Exécuter

l'exemple, puis le modifier pour voir ce qui change.

3 • S'exercer

« Vérifier » corrige tout de suite. Indice, puis solution si besoin.

Variables et types

```
m = 2.5      # float : point décimal
n = 3        # int
nom = "Lou"   # str
ok = n > 2    # bool : True ou False
type(m)      # float
n = n + 1     # nouvelle valeur
```

Opérations

+ - * /	opérations usuelles (7/2 → 3.5)
**	puissance (jamais ^)
// · %	quotient entier · reste
3e8	3×10 ⁸
round(x, 2)	arrondi à 2 décimales

Afficher

```
print("v =", v)
print(f"v = {v:.2f} m/s") # 2 décimales
print(f"N = {N:.3e}")     # 3,000e+08
```

Fonctions

```
def f(x):
    return 3 * x**2 - 2

y = f(4) # 46
```

return renvoie une valeur réutilisable ; print ne fait qu'afficher (la fonction renvoie alors None).

Conditions

```
def nature(pH):
    if pH < 7:
        return "acide"
    elif pH == 7:
        return "neutre"
    else:
        return "basique"
```

== != égal · différent (= affecte)

< <= > >= comparaisons

and or not combiner des conditions

Ligne qui finit par : → bloc décalé de 4 espaces.

Boucle for

<code>range(5)</code>	0, 1, 2, 3, 4 (5 exclu)
<code>range(1, 11)</code>	1 à 10
<code>range(0, 20, 5)</code>	0, 5, 10, 15

Accumulateur

```
S = 0 # initialiser avant
for k in range(1, 101):
    S = S + k # mettre à jour dans
print(S) # 5050
```

Terme d'une suite récurrente

```
#  $u(0) = 2$  et  $u(n+1) = 0,5 u(n) + 3$ 
def u(n):
    terme = 2
    for k in range(n):
        terme = 0.5 * terme + 3
    return terme
```

Boucle while : algorithme de seuil

```
N, n = 1000, 0
while N >= 100: # seuil pas encore franchi
    N = 0.8 * N
    n = n + 1
print(n) # 11 : premier n avec N < 100
```

La condition doit finir par devenir fausse, sinon la boucle est infinie.

Listes

```
L = [12, 15, 9, 18]
L[0], L[-1] # 12 et 18
len(L) # 4 : indices de 0 à 3
L.append(14) # ajoute à la fin
L[1:3] # [15, 9]
sum(L), max(L), min(L)
```

Parcourir

```
for x in L: # les éléments
    print(x)
for i in range(len(L)): # les indices
    print(i, L[i])
```

Listes en compréhension

```
carres = [k**2 for k in range(10)]
pairs = [k for k in range(20) if k % 2 == 0]
moy = sum(L) / len(L)
```

Dichotomie (spé maths)

```
def dichotomie(f, a, b, eps):
    while b - a > eps:
        m = (a + b) / 2
        if f(a) * f(m) <= 0:
            b = m
        else:
            a = m
    return a, b
```

numpy : créer

```
import numpy as np
np.array([2.1, 3.9, 6.2]) # depuis une liste
np.linspace(0, 10, 101)  # 10 inclus
np.arange(0, 1, 0.1)     # 1 exclu
np.zeros(50)             # 50 zéros
```

numpy : calculer sans boucle

```
x = 5 * t**2 # tout le tableau d'un coup
N = N0 * np.exp(-lam * t)
np.sin(x), np.cos(x), np.sqrt(x)
np.log(x), np.exp(x), np.pi
np.mean(x), np.max(x), np.min(x)
```

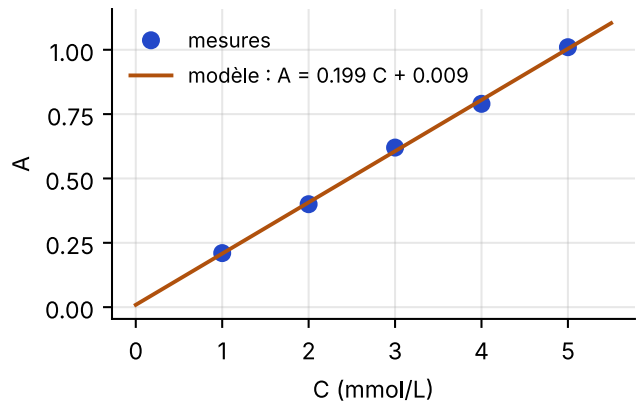
np.diff renvoie une valeur de moins que le tableau de départ. Liste ou tableau ? $[1, 2]*2 \rightarrow [1, 2, 1, 2]$;
 $\text{np.array}([1, 2])*2 \rightarrow [2, 4]$.

numpy : exploiter des mesures

```
v = np.diff(x) / np.diff(t) # vitesses
a = np.diff(v) / np.diff(t[:-1])
k, b = np.polyfit(C, A, 1) # A = k C + b
```

matplotlib : un graphique complet

```
import matplotlib.pyplot as plt
plt.plot(C, A, "o", label="mesures")
plt.plot(C, k*C + b, label="modèle")
plt.xlabel("C (mmol/L)") # unités !
plt.ylabel("A")
plt.title("Loi de Beer-Lambert")
plt.grid()
plt.legend()
plt.show()
```



Vecteurs (vitesse, accélération)

```
plt.quiver(x[:-1], y[:-1], vx, vy,
           angles="xy", scale_units="xy",
           scale=8)
plt.axis("equal")
```

"o" " - " " - - " "o - " points, trait, tirets, les deux

plt.scatter(x, y) nuage de points

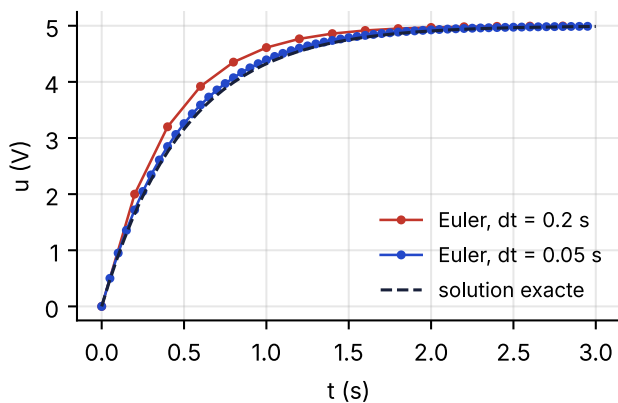
plt.xlim(0, 5) bornes d'un axe

Méthode d'Euler

Pour $dy/dt = F(y)$, on avance par pas dt : $y_{i+1} = y_i + dt \times F(y_i)$.

Charge d'un condensateur : $du/dt = (E - u) / \tau$

```
import numpy as np
E, tau, dt = 5, 0.5, 0.01
t = np.arange(0, 3, dt)
u = np.zeros(len(t))
u[0] = 0 # valeur initiale
for i in range(len(t) - 1):
    u[i + 1] = u[i] + dt * (E - u[i]) / tau
```



Plus dt est petit, plus la courbe d'Euler colle à la solution exacte.

Lire un message d'erreur

Python indique le **type** d'erreur et la **ligne**. L'erreur vient parfois de la ligne *juste au-dessus*.

SyntaxError	Deux-points oubliés après <code>def</code> , <code>if</code> , <code>for</code> , <code>while</code> ; parenthèse ou guillemet non fermé.
IndentationError	Le bloc après <code>:</code> n'est pas décalé de 4 espaces, ou décalé de façon irrégulière.
NameError	Nom inconnu : faute de frappe, majuscule, variable définie plus bas, ou <code>import</code> manquant pour <code>np</code> / <code>plt</code> .
TypeError	Texte + nombre, ou fonction appelée avec un mauvais nombre d'arguments.
IndexError	Indice hors de la liste : de 0 à <code>len(L) - 1</code> .
ValueError (tracé)	<code>x</code> et <code>y</code> de tailles différentes, souvent après <code>np.diff</code> : utiliser <code>t[:-1]</code> .
ZeroDivisionError	Division par zéro (par exemple <code>v = x / t</code> avec <code>t[0] = 0</code>).
Résultat None	La fonction affiche avec <code>print</code> au lieu de renvoyer avec <code>return</code> .
Boucle infinie	La condition du <code>while</code> ne devient jamais fausse : la variable testée ne change pas.
Résultat faux, sans erreur	<code>^</code> au lieu de <code>**</code> ; 9,81 au lieu de 9.81 ; <code>range(1, 100)</code> qui s'arrête à 99.

Une solution possible pour chaque exercice ; d'autres écritures sont acceptées par la vérification automatique.

Ex. 0.1 Bonjour terminale

Affiche exactement le texte `Bonjour terminale`.

```
print("Bonjour terminale")
```

Ex. 1.1 Énergie cinétique

Un ballon de masse $m = 0.45$ kg se déplace à $v = 20$ m/s. Calcule son énergie cinétique $E_c = \frac{1}{2} m v^2$ dans une variable `Ec`, puis affiche-la.

```
m = 0.45
v = 20
Ec = 0.5 * m * v**2
print(Ec)
```

Ex. 1.2 Division euclidienne

Range dans `q` le quotient et dans `r` le reste de la division euclidienne de 2026 par 7.

```
q = 2026 // 7
r = 2026 % 7
print(q, r)
```

Ex. 2.1 Une fonction mathématique

Définis la fonction `g` telle que $g(x) = x^2 - 4x + 3$. Elle doit renvoyer le résultat.

```
def g(x):
    return x**2 - 4*x + 3

print(g(1), g(2))
```

Ex. 2.2 Acide, neutre ou basique

À 25 °C, une solution est acide si $\text{pH} < 7$, neutre si $\text{pH} = 7$, basique si $\text{pH} > 7$. Écris la fonction `nature(pH)` qui renvoie "acide", "neutre" ou "basique".

```
def nature(pH):
    if pH < 7:
        return "acide"
    elif pH == 7:
        return "neutre"
    else:
        return "basique"

print(nature(3.2), nature(7), nature(11))
```

Ex. 3.1 Somme de 1 à 100

Avec une boucle `for`, calcule dans `S` la somme $1 + 2 + 3 + \dots + 100$.

```
S = 0
for k in range(1, 101):
    S = S + k
print(S)
```

Ex. 3.2 Terme d'une suite

La suite (v_n) est définie par $v_0 = 0$ et $v_{n+1} = 2v_n + 1$. Écris la fonction `v(n)` qui renvoie v_n .

```
def v(n):
    terme = 0
    for k in range(n):
        terme = 2 * terme + 1
    return terme

print(v(3))
```

Ex. 3.3 Seuil : décroissance radioactive

Un échantillon contient 1000 noyaux ; chaque année, il en reste 80 %. Écris la fonction `seuil(s)` qui renvoie le **premier** nombre d'années n pour lequel il reste **strictement moins** de `s` noyaux.

```
def seuil(s):
    N = 1000
    n = 0
    while N >= s:
        N = 0.8 * N
        n = n + 1
    return n

print(seuil(100))
```

Ex. 4.1 Liste des carrés

Construis la liste `carres` contenant les carrés des entiers de 0 à 9 : [0, 1, 4, ..., 81].

```
carres = [k**2 for k in range(10)]
print(carres)
```

Ex. 4.2 Moyenne d'une série de mesures

Écris la fonction `moyenne(L)` qui renvoie la moyenne des nombres de la liste `L` (sans numpy).

```
def moyenne(L):
    return sum(L) / len(L)

mesures = [9.79, 9.83, 9.80, 9.85, 9.78]
print(moyenne(mesures))
```

Ex. 4.3 Filtrer les valeurs

Écris la fonction `positifs(L)` qui renvoie la liste des éléments **strictement positifs** de `L`, dans le même ordre.

```
def positifs(L):
    return [x for x in L if x > 0]

print(positifs([3, -1, 0, 7, -5, 2]))
```

Ex. 5.1 Loi de décroissance

Crée `t` : 101 instants régulièrement espacés de 0 à 10 s. Calcule ensuite le tableau `N` = $N_0 e^{-\lambda t}$ avec $N_0 = 1000$ et $\lambda = 0,3 \text{ s}^{-1}$.

```
import numpy as np

N0 = 1000
lam = 0.3
t = np.linspace(0, 10, 101)
N = N0 * np.exp(-lam * t)
print(N[0], N[-1])
```

Ex. 5.2 Vitesse à partir des positions

Une chronophotographie donne les positions `x` (m) aux instants `t` (s). Calcule le tableau `v` des vitesses moyennes entre deux images successives.

```
import numpy as np

t = np.array([0, 0.1, 0.2, 0.3, 0.4])
x = np.array([0, 0.05, 0.20, 0.45, 0.80])
v = np.diff(x) / np.diff(t)
print(v)
```

Ex. 5.3 Régression : loi de Beer-Lambert

On mesure l'absorbance `A` de solutions de concentration `C` (mmol/L). Avec `np.polyfit`, range dans `k` le coefficient directeur et dans `b` l'ordonnée à l'origine de la droite de régression de `A` en fonction de `C`.

```
import numpy as np

C = np.array([1.0, 2.0, 3.0, 4.0, 5.0])
A = np.array([0.21, 0.40, 0.62, 0.79, 1.01])
k, b = np.polyfit(C, A, 1)
print(f"A = {k:.3f} * C + {b:.3f}")
```

Ex. 6.1 Tracer la décroissance

Trace $N(t) = 1000 e^{-0,3t}$ pour t entre 0 et 10 s. Nomme les axes : "`t (s)`" en abscisse et "`N`" en ordonnée.

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 10, 101)
N = 1000 * np.exp(-0.3 * t)

plt.plot(t, N)
plt.xlabel("t (s)")
plt.ylabel("N")
plt.show()
```

Ex. 6.2 Mesures et droite de régression

Sur un même graphique : trace les mesures (`C`, `A`) en points avec l'étiquette "`mesures`", puis la droite modèle $A = k C + b$ avec l'étiquette "`modèle`". Affiche la légende.

```
import numpy as np
import matplotlib.pyplot as plt

C = np.array([1.0, 2.0, 3.0, 4.0, 5.0])
A = np.array([0.21, 0.40, 0.62, 0.79, 1.01])
k, b = np.polyfit(C, A, 1)

plt.plot(C, A, "o", label="mesures")
plt.plot(C, k * C + b, label="modèle")
plt.xlabel("C (mmol/L)")
plt.ylabel("A")
plt.legend()
plt.show()
```

Ex. 7.1 Charge d'un condensateur

Complète la boucle pour remplir le tableau `u` par la méthode d'Euler ($E = 5 \text{ V}$, $\tau = 0,5 \text{ s}$, $dt = 0,01 \text{ s}$, $u(0) = 0$). Trace ensuite `u` et la solution exacte sur le même graphique, avec une légende.

```
import numpy as np
import matplotlib.pyplot as plt

E, tau, dt = 5, 0.5, 0.01
t = np.arange(0, 3, dt)
u = np.zeros(len(t))
u[0] = 0
for i in range(len(t) - 1):
    u[i + 1] = u[i] + dt * (E - u[i]) / tau

plt.plot(t, u, label="Euler")
plt.plot(t, E * (1 - np.exp(-t / tau)), "--",
         label="exacte")
plt.xlabel("t (s)")
plt.ylabel("u (V)")
plt.legend()
plt.show()
```

Ex. 7.2 · bonus Pour aller plus loin : dichotomie

La fonction $f(x) = x^3 + x - 1$ s'annule une seule fois sur $[0 ; 1]$. Écris `dichotomie(a, b, eps)` qui renvoie un encadrement (`a`, `b`) de cette racine d'amplitude inférieure ou égale à `eps`.

```
def f(x):
    return x**3 + x - 1

def dichotomie(a, b, eps):
    while b - a > eps:
        m = (a + b) / 2
        if f(a) * f(m) <= 0:
            b = m
        else:
            a = m
    return a, b

print(dichotomie(0, 1, 0.001))
```

Je fais le point

- ☐ Je sais créer une variable et calculer avec `**`, `//`, `%`.
- ☐ Je sais écrire une fonction qui **renvoie** un résultat.
- ☐ Je sais écrire un test avec `if` / `elif` / `else`.
- ☐ Je sais calculer une somme ou le terme d'une suite avec `for`.
- ☐ Je sais écrire un algorithme de seuil avec `while`.
- ☐ Je sais créer, parcourir et construire une liste (compréhension).
- ☐ Je sais créer un tableau numpy et calculer dessus sans boucle.
- ☐ Je sais obtenir des vitesses avec `np.diff` et une droite avec `np.polyfit`.
- ☐ Je sais tracer une courbe légendée, avec des axes nommés et leurs unités.
- ☐ Je sais programmer une méthode d'Euler.